

MYSQL VIEWS

Practical Examples and Student Exercises using MySQL Workbench

1.What is a View?

A **View** is a **virtual table** created from one or more existing tables.

- It **does not store data** itself.
- It stores only the **SQL query**.
- Whenever you query the view, MySQL retrieves the latest data from the underlying tables.

The screenshot displays the MySQL Workbench interface. The SQL editor at the top contains the following code:

```
1 DROP VIEW IF EXISTS BasicEmployeeView;
2 CREATE VIEW BasicEmployeeView AS
3 SELECT emp_id, emp_name, salary
4 FROM employees;
5
6 -- Test to show the result for your screenshot
7 SELECT * FROM BasicEmployeeView;
```

Below the editor, the 'Result Grid' tab is active, showing the output of the last query. It displays a table with 5 rows and 3 columns: emp_id, emp_name, and salary.

emp_id	emp_name	salary
101	Rahul	54000.00
102	Priya	67000.00
103	Anil	55000.00
104	Sneha	71000.00
105	Kiran	48000.00

At the bottom, the 'Output' tab is active, showing a log of database actions. The log includes the following entries:

#	Time	Action	Message
91	12:27:33	SHOW TABLES	3 row(s) returned
92	12:28:00	DESCRIBE employees	4 row(s) returned
93	13:38:49	USE employeeedb1	0 row(s) affected
94	13:39:25	DROP VIEW IF EXISTS BasicEmployeeView	0 row(s) affected, 1 warning(s): 105
95	13:39:25	CREATE VIEW BasicEmployeeView AS SELECT emp_id, emp_name, salary FRO...	0 row(s) affected
96	13:39:25	SELECT * FROM BasicEmployeeView LIMIT 0, 1000	5 row(s) returned

2. Setup: Employees and Departments Tables

Run this setup first. If these tables already exist with data, you may skip the setup.

```
CREATE DATABASE IF NOT EXISTS employeedb1;  
USE employeedb1;
```

```
CREATE TABLE IF NOT EXISTS departments (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(50)  
);
```

```
CREATE TABLE IF NOT EXISTS employees (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50),  
    dept_id INT,  
    salary DECIMAL(10,2)  
);
```

```
INSERT IGNORE INTO departments VALUES  
(1,'CSE'),  
(2,'ECE'),  
(3,'EEE');
```

```
INSERT IGNORE INTO employees VALUES  
(101,'Rahul',1,50000),  
(102,'Priya',2,65000),  
(103,'Anil',1,55000),  
(104,'Sneha',3,70000),  
(105,'Kiran',2,48000);
```

3. Verify the Tables

```
SHOW TABLES;
```

```
SELECT * FROM departments;
```

SELECT * FROM employees;

The screenshot shows a database management tool interface. At the top, there's a toolbar with various icons and a 'Limit to 1000 rows' dropdown. Below the toolbar, a SQL script is displayed with line numbers 17 to 23. The script includes an INSERT statement, a SHOW TABLES command, and two SELECT statements. Below the script, there's a 'Result Grid' section with a table showing the results of the SELECT * FROM employees query. The table has columns emp_id, emp_name, dept_id, and salary. Below the table, there's a 'Result 35' section with tabs for departments 36 and employees 37. At the bottom, there's an 'Output' section with a dropdown menu set to 'Action Output'. The output shows a list of actions and their messages, including warnings and successful results.

```
17  INSERT IGNORE INTO employees VALUES
18  (101,'Rahul',1,50000),(102,'Priya',2,65000),(103,'Anil',1,55000),
19  (104,'Sneha',3,70000),(105,'Kiran',2,48000);
20
21  SHOW TABLES;
22  SELECT * FROM departments;
23  SELECT * FROM employees;
```

emp_id	emp_name	dept_id	salary
101	Rahul	1	54000.00
102	Priya	2	67000.00
103	Anil	2	55000.00
104	Sneha	3	71000.00
105	Kiran	2	48000.00
* NULL	NULL	NULL	NULL

Result 35 departments 36 employees 37 x

Output:

#	Time	Action	Message
115	13:44:00	CREATE TABLE IF NOT EXISTS employees (emp_id INT PRIMARY KEY, e...	0 row(s) affected, 1 warning(s): 1050 Table 'em
116	13:44:00	INSERT IGNORE INTO departments VALUES(1,'CSE'),(2,'ECE'),(3,'EEE')	0 row(s) affected, 3 warning(s): 1062 Duplicate
117	13:44:00	INSERT IGNORE INTO employees VALUES (101,'Rahul',1,50000),(102,'Priya',2,65...	0 row(s) affected, 5 warning(s): 1062 Duplicate
118	13:44:00	SHOW TABLES	5 row(s) returned
119	13:44:00	SELECT * FROM departments LIMIT 0, 1000	3 row(s) returned
120	13:44:00	SELECT * FROM employees LIMIT 0, 1000	5 row(s) returned

View 1 – Display All Employees

Question: Create a View named EmployeeView to display all columns from employees.

DROP VIEW IF EXISTS EmployeeView;

```
CREATE VIEW EmployeeView AS
SELECT *
FROM employees;
```

SELECT * FROM EmployeeView;

The screenshot shows a database IDE interface. The top toolbar includes icons for file operations, execution, and a 'Limit to 1000 rows' dropdown. The SQL editor contains the following script:

```
1 • DROP VIEW IF EXISTS EmployeeView;
2 CREATE VIEW EmployeeView AS
3 SELECT * FROM employees;
4
5 • SELECT * FROM EmployeeView;
```

Below the editor is the 'Result Grid' section, which displays a table with 5 rows and 4 columns: emp_id, emp_name, dept_id, and salary.

emp_id	emp_name	dept_id	salary
101	Rahul	1	54000.00
102	Priya	2	67000.00
103	Anil	2	55000.00
104	Sneha	3	71000.00
105	Kiran	2	48000.00

Below the result grid is the 'Output' section, which shows the 'Action Output' log. It contains a table with columns: #, Time, Action, and Message.

#	Time	Action	Message
✓ 118	13:44:00	SHOW TABLES	5 row(s) returned
✓ 119	13:44:00	SELECT * FROM departments LIMIT 0, 1000	3 row(s) returned
✓ 120	13:44:00	SELECT * FROM employees LIMIT 0, 1000	5 row(s) returned
⚠ 121	13:44:26	DROP VIEW IF EXISTS EmployeeView	0 row(s) affected, 1 warning(s): 1051 Unknown
✓ 122	13:44:26	CREATE VIEW EmployeeView AS SELECT * FROM employees	0 row(s) affected
✓ 123	13:44:26	SELECT * FROM EmployeeView LIMIT 0, 1000	5 row(s) returned

View 2 – Selected Columns

Question: Create a View showing employee ID, employee name and salary only.

DROP VIEW IF EXISTS EmployeeBasicView;

```
CREATE VIEW EmployeeBasicView AS
SELECT emp_id, emp_name, salary
FROM employees;
```

SELECT * FROM EmployeeBasicView;

Query 1 x

Limit to 1000 rows

```

1 • DROP VIEW IF EXISTS EmployeeBasicView;
2 CREATE VIEW EmployeeBasicView AS
3 SELECT emp_id, emp_name, salary
4 FROM employees;
5
6 • SELECT * FROM EmployeeBasicView;

```

Result Grid

emp_id	emp_name	salary
101	Rahul	54000.00
102	Priya	67000.00
103	Anil	55000.00
104	Sneha	71000.00
105	Kiran	48000.00

EmployeeBasicView 39 x

Output

Action Output

#	Time	Action	Message
121	13:44:26	DROP VIEW IF EXISTS EmployeeView	0 row(s) affected, 1 warning(s): 1051 Unknown
122	13:44:26	CREATE VIEW EmployeeView AS SELECT * FROM employees	0 row(s) affected
123	13:44:26	SELECT * FROM EmployeeView LIMIT 0, 1000	5 row(s) returned
124	13:44:51	DROP VIEW IF EXISTS EmployeeBasicView	0 row(s) affected, 1 warning(s): 1051 Unknown
125	13:44:51	CREATE VIEW EmployeeBasicView AS SELECT emp_id, emp_name, salary FRO...	0 row(s) affected
126	13:44:51	SELECT * FROM EmployeeBasicView LIMIT 0, 1000	5 row(s) returned

View 3 – High Salary Employees

Question: Create a View that displays employees earning more than 55000.

DROP VIEW IF EXISTS HighSalaryEmployees;

```

CREATE VIEW HighSalaryEmployees AS
SELECT emp_id, emp_name, salary
FROM employees
WHERE salary > 55000;

```

SELECT * FROM HighSalaryEmployees;

The screenshot shows a SQL IDE interface. The script editor contains the following SQL code:

```
1 • DROP VIEW IF EXISTS HighSalaryEmployees;
2   C
3   SELECT emp_id, emp_name, salary
4   FROM employees
5   WHERE salary > 55000;
6
7 • SELECT * FROM HighSalaryEmployees;
```

A tooltip for line 2 says: "Execute the selected portion of the script or everything, if there is no selection".

Below the script editor is a "Result Grid" showing the output of the last query:

emp_id	emp_name	salary
102	Priya	67000.00
104	Sneha	71000.00

Below the result grid is an "Output" pane showing the "Action Output" log:

#	Time	Action	Message
124	13:44:51	DROP VIEW IF EXISTS EmployeeBasicView	0 row(s) affected, 1 warning(s): 1051 Unknown
125	13:44:51	CREATE VIEW EmployeeBasicView AS SELECT emp_id, emp_name, salary FRO...	0 row(s) affected
126	13:44:51	SELECT * FROM EmployeeBasicView LIMIT 0, 1000	5 row(s) returned
127	13:45:18	DROP VIEW IF EXISTS HighSalaryEmployees	0 row(s) affected, 1 warning(s): 1051 Unknown
128	13:45:18	CREATE VIEW HighSalaryEmployees AS SELECT emp_id, emp_name, salary FRO...	0 row(s) affected
129	13:45:18	SELECT * FROM HighSalaryEmployees LIMIT 0, 1000	2 row(s) returned

View 4 – Employee and Department using JOIN

Question: Create a View that displays employee name, department name and salary.

DROP VIEW IF EXISTS EmployeeDepartmentView;

```
CREATE VIEW EmployeeDepartmentView AS
SELECT
    e.emp_id,
    e.emp_name,
    d.dept_name,
    e.salary
FROM employees e
JOIN departments d
```


ON e.dept_id = d.dept_id;

SELECT * FROM EmployeeDepartmentView;

The screenshot displays the SQL Developer interface. The top pane shows a SQL script with the following steps:

- 1 • DROP VIEW IF EXISTS EmployeeDepartmentView;
- 2 CREATE VIEW EmployeeDepartmentView AS
- 3 SELECT e.emp_id, e.emp_name, d.dept_name, e.salary
- 4 FROM employees e
- 5 JOIN departments d ON e.dept_id = d.dept_id;
- 6
- 7 SELECT * FROM EmployeeDepartmentView;

The bottom pane shows the 'Result Grid' with the following data:

emp_id	emp_name	dept_name	salary
101	Rahul	CSE	54000.00
102	Priya	ECE	67000.00
103	Anil	ECE	55000.00
104	Sneha	EEE	71000.00
105	Kiran	ECE	48000.00

Below the result grid, the 'Output' pane shows the 'Action Output' table:

#	Time	Action	Message
127	13:45:18	DROP VIEW IF EXISTS HighSalaryEmployees	0 row(s) affected, 1 warning(s): 1051 Unknown
128	13:45:18	CREATE VIEW HighSalaryEmployees AS SELECT emp_id, emp_name, salary FRO...	0 row(s) affected
129	13:45:18	SELECT * FROM HighSalaryEmployees LIMIT 0, 1000	2 row(s) returned
130	13:45:43	DROP VIEW IF EXISTS EmployeeDepartmentView	0 row(s) affected, 1 warning(s): 1051 Unknown
131	13:45:43	CREATE VIEW EmployeeDepartmentView AS SELECT e.emp_id, e.emp_name, d....	0 row(s) affected
132	13:45:43	SELECT * FROM EmployeeDepartmentView LIMIT 0, 1000	5 row(s) returned

View 5 – Employees from a Particular Department

Question: Create a View that displays only CSE employees.

DROP VIEW IF EXISTS CSEEmployees;

CREATE VIEW CSEEmployees AS

SELECT

e.emp_id,

e.emp_name,

d.dept_name,

e.salary

```

FROM employees e
JOIN departments d
ON e.dept_id = d.dept_id
WHERE d.dept_name = 'CSE';

```

```

SELECT * FROM CSEEmployees;

```

The screenshot shows the SQL Developer interface. The top pane contains the following SQL code:

```

2 CREATE VIEW CSEEmployees AS
3 SELECT e.emp_id, e.emp_name, d.dept_name, e.salary
4 FROM employees e
5 JOIN departments d ON e.dept_id = d.dept_id
6 WHERE d.dept_name = 'CSE';
7
8 • SELECT * FROM CSEEmployees;

```

The bottom pane shows the 'Result Grid' with the following data:

emp_id	emp_name	dept_name	salary
101	Rahul	CSE	54000.00

Below the result grid, the 'Output' pane shows the 'Action Output' with the following log entries:

#	Time	Action	Message
130	13:45:43	DROP VIEW IF EXISTS EmployeeDepartmentView	0 row(s) affected, 1 warning(s): 1051 Unknown
131	13:45:43	CREATE VIEW EmployeeDepartmentView AS SELECT e.emp_id, e.emp_name, d....	0 row(s) affected
132	13:45:43	SELECT * FROM EmployeeDepartmentView LIMIT 0, 1000	5 row(s) returned
133	13:46:16	DROP VIEW IF EXISTS CSEEmployees	0 row(s) affected, 1 warning(s): 1051 Unknown
134	13:46:16	CREATE VIEW CSEEmployees AS SELECT e.emp_id, e.emp_name, d.dept_name,...	0 row(s) affected
135	13:46:16	SELECT * FROM CSEEmployees LIMIT 0, 1000	1 row(s) returned

View 6 – Department-wise Average Salary

Question: Create a View using GROUP BY and AVG() to display department-wise average salary.

```

DROP VIEW IF EXISTS DepartmentSalaryView;

```

```

CREATE VIEW DepartmentSalaryView AS
SELECT
    d.dept_name,

```

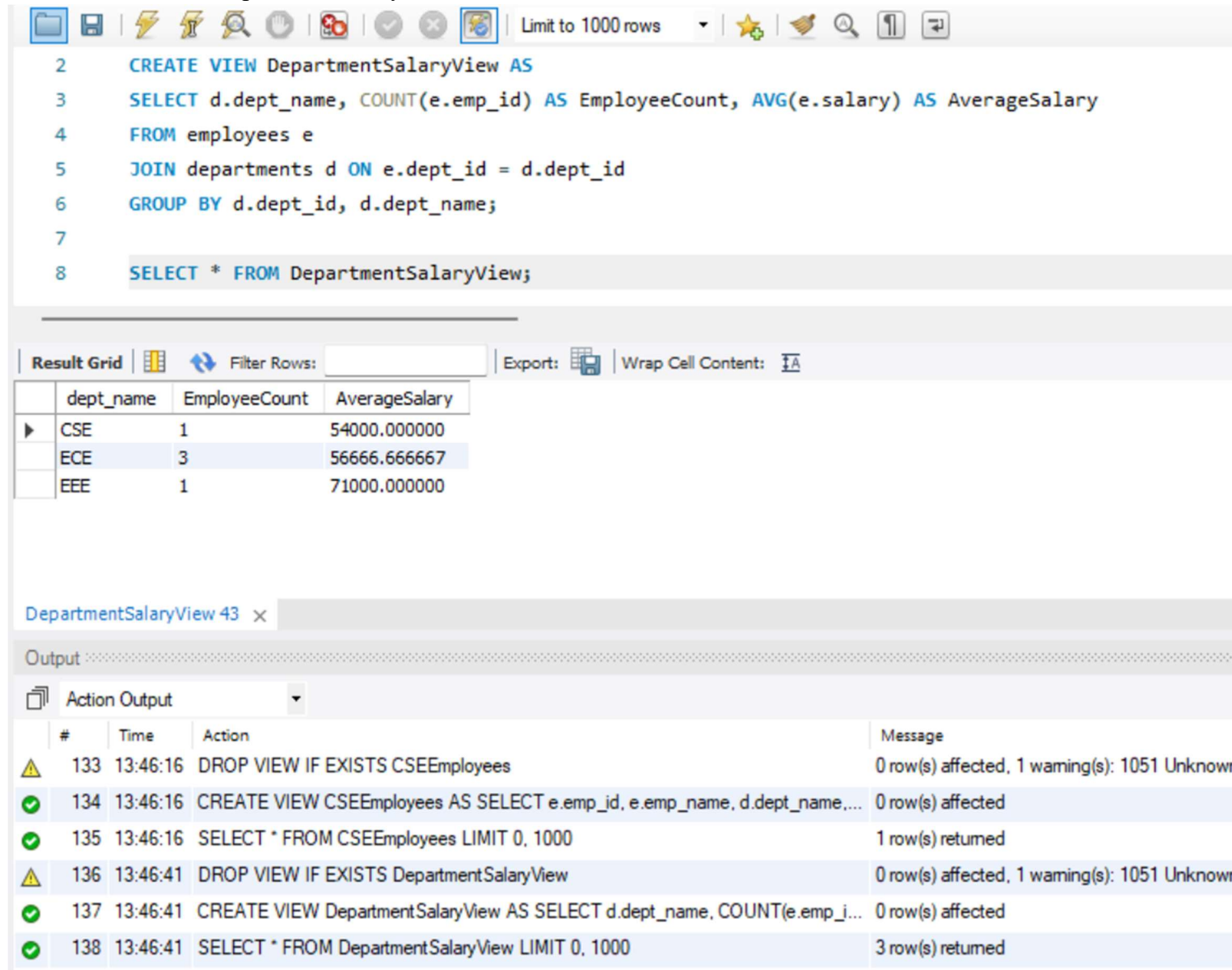


```

COUNT(e.emp_id) AS EmployeeCount,
AVG(e.salary) AS AverageSalary
FROM employees e
JOIN departments d
ON e.dept_id = d.dept_id
GROUP BY d.dept_id, d.dept_name;

```

```
SELECT * FROM DepartmentSalaryView;
```



SQL Editor:

```

2 CREATE VIEW DepartmentSalaryView AS
3 SELECT d.dept_name, COUNT(e.emp_id) AS EmployeeCount, AVG(e.salary) AS AverageSalary
4 FROM employees e
5 JOIN departments d ON e.dept_id = d.dept_id
6 GROUP BY d.dept_id, d.dept_name;
7
8 SELECT * FROM DepartmentSalaryView;

```

Results Grid:

	dept_name	EmployeeCount	AverageSalary
▶	CSE	1	54000.000000
	ECE	3	56666.666667
	EEE	1	71000.000000

Output:

#	Time	Action	Message
133	13:46:16	DROP VIEW IF EXISTS CSEEmployees	0 row(s) affected, 1 warning(s): 1051 Unknown
134	13:46:16	CREATE VIEW CSEEmployees AS SELECT e.emp_id, e.emp_name, d.dept_name,...	0 row(s) affected
135	13:46:16	SELECT * FROM CSEEmployees LIMIT 0, 1000	1 row(s) returned
136	13:46:41	DROP VIEW IF EXISTS DepartmentSalaryView	0 row(s) affected, 1 warning(s): 1051 Unknown
137	13:46:41	CREATE VIEW DepartmentSalaryView AS SELECT d.dept_name, COUNT(e.emp_i...	0 row(s) affected
138	13:46:41	SELECT * FROM DepartmentSalaryView LIMIT 0, 1000	3 row(s) returned

View 7 – Departments with Average Salary Above 55000

Question: Create a View that displays departments whose average salary is greater than 55000.

```
DROP VIEW IF EXISTS HighAverageDepartments;
```

```
CREATE VIEW HighAverageDepartments AS
```

```

SELECT
    d.dept_name,
    AVG(e.salary) AS AverageSalary
FROM employees e
JOIN departments d
ON e.dept_id = d.dept_id
GROUP BY d.dept_id, d.dept_name
HAVING AVG(e.salary) > 55000;

```

```

SELECT * FROM HighAverageDepartments;

```

The screenshot shows the SQL Developer interface. The top pane displays the following SQL queries:

```

3  SELECT d.dept_name, AVG(e.salary) AS AverageSalary
4  FROM employees e
5  JOIN departments d ON e.dept_id = d.dept_id
6  GROUP BY d.dept_id, d.dept_name
7  HAVING AVG(e.salary) > 55000;
8
9  • SELECT * FROM HighAverageDepartments;

```

The bottom pane shows the 'Result Grid' with the following data:

dept_name	AverageSalary
ECE	56666.666667
EEE	71000.000000

Below the result grid, the 'Output' pane shows the 'Action Output' log with the following entries:

#	Time	Action	Message
136	13:46:41	DROP VIEW IF EXISTS DepartmentSalaryView	0 row(s) affected, 1 warning(s): 1051 Unknown table or view
137	13:46:41	CREATE VIEW DepartmentSalaryView AS SELECT d.dept_name, COUNT(e.emp_id) AS EmployeeCount FROM employees e JOIN departments d ON e.dept_id = d.dept_id	0 row(s) affected
138	13:46:41	SELECT * FROM DepartmentSalaryView LIMIT 0, 1000	3 row(s) returned
139	13:47:06	DROP VIEW IF EXISTS HighAverageDepartments	0 row(s) affected, 1 warning(s): 1051 Unknown table or view
140	13:47:06	CREATE VIEW HighAverageDepartments AS SELECT d.dept_name, AVG(e.salary) AS AverageSalary FROM employees e JOIN departments d ON e.dept_id = d.dept_id GROUP BY d.dept_id, d.dept_name HAVING AVG(e.salary) > 55000	0 row(s) affected
141	13:47:06	SELECT * FROM HighAverageDepartments LIMIT 0, 1000	2 row(s) returned

View 8 – Salary Range View

Question: Create a View that displays employees with salary between 50000 and 70000.

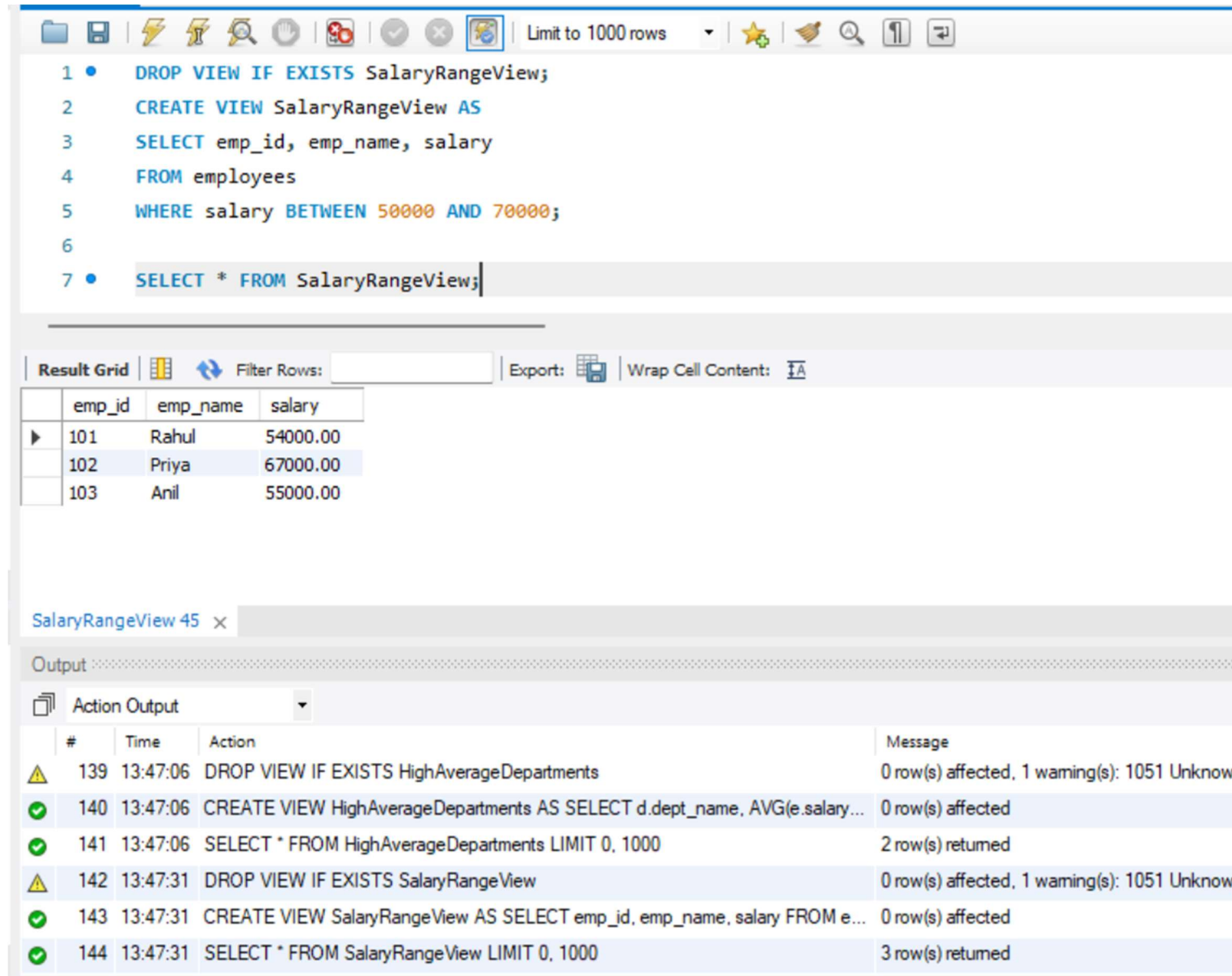
```

DROP VIEW IF EXISTS SalaryRangeView;

```

```
CREATE VIEW SalaryRangeView AS
SELECT emp_id, emp_name, salary
FROM employees
WHERE salary BETWEEN 50000 AND 70000;
```

```
SELECT * FROM SalaryRangeView;
```



The screenshot displays a SQL IDE interface. The top pane shows the SQL script being executed:

```
1 • DROP VIEW IF EXISTS SalaryRangeView;
2 CREATE VIEW SalaryRangeView AS
3 SELECT emp_id, emp_name, salary
4 FROM employees
5 WHERE salary BETWEEN 50000 AND 70000;
6
7 • SELECT * FROM SalaryRangeView;
```

The middle pane shows the 'Result Grid' with the following data:

emp_id	emp_name	salary
101	Rahul	54000.00
102	Priya	67000.00
103	Anil	55000.00

The bottom pane shows the 'Output' window with the 'Action Output' tab selected, displaying the execution log:

#	Time	Action	Message
139	13:47:06	DROP VIEW IF EXISTS HighAverageDepartments	0 row(s) affected, 1 warning(s): 1051 Unknown
140	13:47:06	CREATE VIEW HighAverageDepartments AS SELECT d.dept_name, AVG(e.salary...	0 row(s) affected
141	13:47:06	SELECT * FROM HighAverageDepartments LIMIT 0, 1000	2 row(s) returned
142	13:47:31	DROP VIEW IF EXISTS SalaryRangeView	0 row(s) affected, 1 warning(s): 1051 Unknown
143	13:47:31	CREATE VIEW SalaryRangeView AS SELECT emp_id, emp_name, salary FROM e...	0 row(s) affected
144	13:47:31	SELECT * FROM SalaryRangeView LIMIT 0, 1000	3 row(s) returned

View 9 – View with Calculated Column

Question: Create a View that displays employee name, monthly salary and annual salary.

```
DROP VIEW IF EXISTS AnnualSalaryView;
```

```
CREATE VIEW AnnualSalaryView AS
SELECT
    emp_id,
```

```

emp_name,
salary AS MonthlySalary,
salary * 12 AS AnnualSalary
FROM employees;

```

```

SELECT * FROM AnnualSalaryView;

```

The screenshot shows a database IDE with a SQL editor and a results pane. The SQL editor contains the following code:

```

1 • DROP VIEW IF EXISTS AnnualSalaryView;
2   CREATE VIEW AnnualSalaryView AS
3   SELECT emp_id, emp_name, salary AS MonthlySalary, salary * 12 AS AnnualSalary
4   FROM employees;
5
6 • SELECT * FROM AnnualSalaryView;

```

The results pane displays a table with 5 rows and 4 columns: emp_id, emp_name, MonthlySalary, and AnnualSalary.

emp_id	emp_name	MonthlySalary	AnnualSalary
101	Rahul	54000.00	648000.00
102	Priya	67000.00	804000.00
103	Anil	55000.00	660000.00
104	Sneha	71000.00	852000.00
105	Kiran	48000.00	576000.00

The output pane shows the execution log with the following entries:

#	Time	Action	Message
142	13:47:31	DROP VIEW IF EXISTS SalaryRangeView	0 row(s) affected, 1 warning(s): 1051 Unknown table or view name 'SalaryRangeView'
143	13:47:31	CREATE VIEW SalaryRangeView AS SELECT emp_id, emp_name, salary FROM employees	0 row(s) affected
144	13:47:31	SELECT * FROM SalaryRangeView LIMIT 0, 1000	3 row(s) returned
145	13:47:58	DROP VIEW IF EXISTS AnnualSalaryView	0 row(s) affected, 1 warning(s): 1051 Unknown table or view name 'AnnualSalaryView'
146	13:47:58	CREATE VIEW AnnualSalaryView AS SELECT emp_id, emp_name, salary AS MonthlySalary, salary * 12 AS AnnualSalary FROM employees	0 row(s) affected
147	13:47:58	SELECT * FROM AnnualSalaryView LIMIT 0, 1000	5 row(s) returned

View 10 – Update Data Through a Simple View

Question: Create a simple single-table View and update an employee's salary through the View.

```

DROP VIEW IF EXISTS EmployeeSalaryView;

```

```

CREATE VIEW EmployeeSalaryView AS
SELECT emp_id, emp_name, salary
FROM employees;

```



```
UPDATE EmployeeSalaryView
SET salary = 60000
WHERE emp_id = 101;
```

```
SELECT * FROM EmployeeSalaryView;
SELECT * FROM employees WHERE emp_id = 101;
```

The screenshot shows the SQL Developer interface. The 'Query' window contains the following SQL commands:

```
5
6 UPDATE EmployeeSalaryView
7 SET salary = 60000
8 WHERE emp_id = 101;
9
10 • SELECT * FROM EmployeeSalaryView;
11 • SELECT * FROM employees WHERE emp_id = 101;
```

Below the query window is the 'Result Grid' showing the results of the queries. The first query (SELECT * FROM EmployeeSalaryView) returned 5 rows. The second query (SELECT * FROM employees WHERE emp_id = 101) returned 1 row.

emp_id	emp_name	dept_id	salary
101	Rahul	1	60000.00
NULL	NULL	NULL	NULL

The 'Output' window shows the 'Action Output' for the queries:

#	Time	Action	Message
147	13:47:58	SELECT * FROM AnnualSalaryView LIMIT 0, 1000	5 row(s) returned
148	13:48:22	DROP VIEW IF EXISTS EmployeeSalaryView	0 row(s) affected, 1 warning(s): 1051 Unknown table or view
149	13:48:22	CREATE VIEW EmployeeSalaryView AS SELECT emp_id, emp_name, salary FROM employees	0 row(s) affected
150	13:48:22	UPDATE EmployeeSalaryView SET salary = 60000 WHERE emp_id = 101	1 row(s) affected Rows matched: 1 Changed: 1
151	13:48:22	SELECT * FROM EmployeeSalaryView LIMIT 0, 1000	5 row(s) returned
152	13:48:23	SELECT * FROM employees WHERE emp_id = 101 LIMIT 0, 1000	1 row(s) returned

View 11 – CREATE OR REPLACE VIEW

Question: Modify an existing View so that it displays employee name and salary only.

```
CREATE OR REPLACE VIEW EmployeeBasicView AS
SELECT emp_name, salary
FROM employees;
```


SELECT * FROM EmployeeBasicView;

The screenshot displays a database management interface with a query editor and a results pane. The query editor contains the following SQL statements:

```
1 • CREATE OR REPLACE VIEW EmployeeBasicView AS
2   SELECT emp_name, salary
3   FROM employees;
4
5 • SELECT * FROM EmployeeBasicView;
```

The results pane shows a table with two columns: `emp_name` and `salary`. The data is as follows:

emp_name	salary
Rahul	60000.00
Priya	67000.00
Anil	55000.00
Sneha	71000.00
Kiran	48000.00

The output pane shows the execution log with the following entries:

#	Time	Action	Message
✓ 149	13:48:22	CREATE VIEW EmployeeSalaryView AS SELECT emp_id, emp_name, salary FRO...	0 row(s) affected
✓ 150	13:48:22	UPDATE EmployeeSalaryView SET salary = 60000 WHERE emp_id = 101	1 row(s) affected Rows matched: 1 Changed
✓ 151	13:48:22	SELECT * FROM EmployeeSalaryView LIMIT 0, 1000	5 row(s) returned
✓ 152	13:48:23	SELECT * FROM employees WHERE emp_id = 101 LIMIT 0, 1000	1 row(s) returned
✓ 153	13:49:07	CREATE OR REPLACE VIEW EmployeeBasicView AS SELECT emp_name, salary...	0 row(s) affected
✓ 154	13:49:07	SELECT * FROM EmployeeBasicView LIMIT 0, 1000	5 row(s) returned

View 12 – View Definition

Question: Display the SQL statement used to create a View.

SHOW CREATE VIEW EmployeeDepartmentView;

The screenshot shows a database management tool interface. At the top, a toolbar contains various icons for file operations, execution, and navigation. Below the toolbar, a text area contains the SQL command: `SHOW CREATE VIEW EmployeeDepartmentView;`. The command is numbered 1. Below the text area, a "Result Grid" is displayed. It has a header row with columns: "View", "Create View", "character_set_client", and "collation_connection". The first row of data shows the results for the command: "employeedepartmentview", "CREATE ALGORITHM=UNDEFINED DEFINER=`r...`", "utf8mb4", and "utf8mb4_0900_ai_ci". Below the result grid, there is an "Output" section. It has a tab labeled "Action Output". The output is a table with columns: "#", "Time", "Action", and "Message". It contains six rows of execution logs, each with a green checkmark in the first column. The last row shows the command being executed: "SHOW CREATE VIEW EmployeeDepartmentView" and the message "1 row(s) returned".

View	Create View	character_set_client	collation_connection
employeedepartmentview	CREATE ALGORITHM=UNDEFINED DEFINER=`r...`	utf8mb4	utf8mb4_0900_ai_ci

#	Time	Action	Message
✓ 150	13:48:22	UPDATE EmployeeSalaryView SET salary = 60000 WHERE emp_id = 101	1 row(s) affected Rows matched: 1 Change
✓ 151	13:48:22	SELECT * FROM EmployeeSalaryView LIMIT 0, 1000	5 row(s) returned
✓ 152	13:48:23	SELECT * FROM employees WHERE emp_id = 101 LIMIT 0, 1000	1 row(s) returned
✓ 153	13:49:07	CREATE OR REPLACE VIEW EmployeeBasicView AS SELECT emp_name, salary...	0 row(s) affected
✓ 154	13:49:07	SELECT * FROM EmployeeBasicView LIMIT 0, 1000	5 row(s) returned
✓ 155	13:49:30	SHOW CREATE VIEW EmployeeDepartmentView	1 row(s) returned

View 13 – List All Views

Question: Display all Views available in the current database.

SHOW FULL TABLES

WHERE Table_type = 'VIEW';

The screenshot shows a database management tool interface. At the top, there is a toolbar with various icons and a dropdown menu set to 'Limit to 1000 rows'. Below the toolbar, the SQL editor contains two lines of code:

```
1 SHOW FULL TABLES
2 WHERE Table_type = 'VIEW';
```

Below the editor, the 'Result Grid' tab is active, displaying a table with two columns: 'Tables_in_employeedb1' and 'Table_type'. The table lists several views:

Tables_in_employeedb1	Table_type
annualsalaryview	VIEW
basicemployeeview	VIEW
cseemployees	VIEW
departmentsalaryview	VIEW
employeebasicview	VIEW
employeeedepartmentview	VIEW
employeeesalaryview	VIEW

Below the result grid, the 'Output' tab is active, showing a log of actions and their results:

#	Time	Action	Message
✓ 151	13:48:22	SELECT * FROM EmployeeSalaryView LIMIT 0, 1000	5 row(s) returned
✓ 152	13:48:23	SELECT * FROM employees WHERE emp_id = 101 LIMIT 0, 1000	1 row(s) returned
✓ 153	13:49:07	CREATE OR REPLACE VIEW EmployeeBasicView AS SELECT emp_name, salary...	0 row(s) affected
✓ 154	13:49:07	SELECT * FROM EmployeeBasicView LIMIT 0, 1000	5 row(s) returned
✓ 155	13:49:30	SHOW CREATE VIEW EmployeeDepartmentView	1 row(s) returned
✓ 156	13:50:02	SHOW FULL TABLES WHERE Table_type = 'VIEW'	12 row(s) returned

View 14 – Drop a View

Question: Remove an existing View from the database.

DROP VIEW IF EXISTS SalaryRangeView;

The screenshot shows a database query editor window titled 'Query 1'. The query text is 'DROP VIEW IF EXISTS SalaryRangeView;'. Below the query editor is an 'Output' window showing a table of execution results. The table has columns: #, Time, Action, and Message. The results show a sequence of operations including a SELECT query, creating or replacing a view, another SELECT query, showing the view's definition, showing full tables, and finally dropping the view.

#	Time	Action	Message
152	13:48:23	SELECT * FROM employees WHERE emp_id = 101 LIMIT 0, 1000	1 row(s) returned
153	13:49:07	CREATE OR REPLACE VIEW EmployeeBasicView AS SELECT emp_name, salary...	0 row(s) affected
154	13:49:07	SELECT * FROM EmployeeBasicView LIMIT 0, 1000	5 row(s) returned
155	13:49:30	SHOW CREATE VIEW EmployeeDepartmentView	1 row(s) returned
156	13:50:02	SHOW FULL TABLES WHERE Table_type = 'VIEW'	12 row(s) returned
157	13:50:36	DROP VIEW IF EXISTS SalaryRangeView	0 row(s) affected

4. Important Concept: Simple View vs Complex View

A simple View is generally based on one table and does not use grouping or aggregate functions. Some simple Views can be updated. A View containing JOINS, GROUP BY, aggregate functions such as AVG(), DISTINCT, or other non-updatable constructs may not be directly updatable.

5. View vs Table

Feature	Table	View
Stores data	Yes	Normally no; stores query definition
Created with CREATE TABLE	Yes	No
Created with CREATE VIEW	No	Yes
Can use JOINS	Not applicable	Yes

Reflects current underlying data	Own stored data	Yes
Can always be updated	Yes	No; depends on the View

6. Student Practice Tasks

1. Create a View showing only emp_id, emp_name and salary.

Query 1 x

```

1 DROP VIEW IF EXISTS BasicEmployeeView;
2 CREATE VIEW BasicEmployeeView AS
3 SELECT emp_id, emp_name, salary
4 FROM employees;
5
6 -- Run this to show the output for your screenshot:
7 SELECT * FROM BasicEmployeeView;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	emp_id	emp_name	salary
▶	101	Rahul	60000.00
	102	Priya	67000.00
	103	Anil	55000.00
	104	Sneha	71000.00
	105	Kiran	48000.00

BasicEmployeeView 52 x

Output

Action Output

#	Time	Action	Message
✓ 155	13:49:30	SHOW CREATE VIEW EmployeeDepartmentView	1 row(s) returned
✓ 156	13:50:02	SHOW FULL TABLES WHERE Table_type = 'VIEW'	12 row(s) returned
✓ 157	13:50:36	DROP VIEW IF EXISTS SalaryRangeView	0 row(s) affected
✓ 158	14:00:48	DROP VIEW IF EXISTS BasicEmployeeView	0 row(s) affected
✓ 159	14:00:48	CREATE VIEW BasicEmployeeView AS SELECT emp_id, emp_name, salary FRO...	0 row(s) affected
✓ 160	14:00:48	SELECT * FROM BasicEmployeeView LIMIT 0, 1000	5 row(s) returned

2. Create a View for employees earning more than 60000.

The screenshot shows a database IDE interface. At the top, a toolbar contains various icons for file operations, execution, and search. Below the toolbar, a SQL editor displays the following code:

```
1 • DROP VIEW IF EXISTS HighEarnerView;
2 CREATE VIEW HighEarnerView AS
3 SELECT *
4 FROM employees
5 WHERE salary > 60000;
6
7 SELECT * FROM HighEarnerView;
```

Below the editor, a "Result Grid" section shows the results of the last query. It includes a "Filter Rows" input field, an "Export" button, and a "Wrap Cell Content" checkbox. The results are displayed in a table with four columns: emp_id, emp_name, dept_id, and salary.

emp_id	emp_name	dept_id	salary
102	Priya	2	67000.00
104	Sneha	3	71000.00

Below the result grid, a tab labeled "HighEarnerView 53" is visible. At the bottom, an "Output" section shows the "Action Output" log, which records the execution of the SQL statements and their results.

#	Time	Action	Message
✓ 158	14:00:48	DROP VIEW IF EXISTS BasicEmployeeView	0 row(s) affected
✓ 159	14:00:48	CREATE VIEW BasicEmployeeView AS SELECT emp_id, emp_name, salary FRO...	0 row(s) affected
✓ 160	14:00:48	SELECT * FROM BasicEmployeeView LIMIT 0, 1000	5 row(s) returned
✓ 161	14:01:06	DROP VIEW IF EXISTS HighEamerView	0 row(s) affected
✓ 162	14:01:06	CREATE VIEW HighEamerView AS SELECT * FROM employees WHERE salary ...	0 row(s) affected
✓ 163	14:01:06	SELECT * FROM HighEamerView LIMIT 0, 1000	2 row(s) returned

3. Create a View showing employee name, department name and salary using JOIN.

The screenshot displays the SQL Developer interface. The top pane shows a SQL script with the following commands:

```
1 • DROP VIEW IF EXISTS EmpDeptDetailsView;  
2 CREATE VIEW EmpDeptDetailsView AS  
3 SELECT e.emp_name, d.dept_name, e.salary  
4 FROM employees e  
5 JOIN departments d ON e.dept_id = d.dept_id;  
6  
7 SELECT * FROM EmpDeptDetailsView;
```

The bottom pane shows the 'Result Grid' with the following data:

	emp_name	dept_name	salary
▶	Rahul	CSE	60000.00
	Priya	ECE	67000.00
	Anil	ECE	55000.00
	Sneha	EEE	71000.00
	Kiran	ECE	48000.00

Below the result grid, the 'Output' pane shows the 'Action Output' table:

#	Time	Action	Message
✓ 161	14:01:06	DROP VIEW IF EXISTS HighEamerView	0 row(s) affected
✓ 162	14:01:06	CREATE VIEW HighEamerView AS SELECT * FROM employees WHERE salary ...	0 row(s) affected
✓ 163	14:01:06	SELECT * FROM HighEamerView LIMIT 0, 1000	2 row(s) returned
⚠ 164	14:01:23	DROP VIEW IF EXISTS EmpDeptDetailsView	0 row(s) affected, 1 warning(s): 1051 Unknown table or view name
✓ 165	14:01:23	CREATE VIEW EmpDeptDetailsView AS SELECT e.emp_name, d.dept_name, e.s...	0 row(s) affected
✓ 166	14:01:23	SELECT * FROM EmpDeptDetailsView LIMIT 0, 1000	5 row(s) returned

4. Create a View showing only ECE employees.

```
2 CREATE VIEW ECEEmployeesView AS
3 SELECT e.emp_id, e.emp_name, d.dept_name, e.salary
4 FROM employees e
5 JOIN departments d ON e.dept_id = d.dept_id
6 WHERE d.dept_name = 'ECE';
7
8 • SELECT * FROM ECEEmployeesView;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	emp_id	emp_name	dept_name	salary
▶	102	Priya	ECE	67000.00
	103	Anil	ECE	55000.00
	105	Kiran	ECE	48000.00

ECEEmployeesView 55 x

Output

Action Output

#	Time	Action	Message
⚠ 164	14:01:23	DROP VIEW IF EXISTS EmpDeptDetailsView	0 row(s) affected, 1 warning(s): 1051 Unknown
✅ 165	14:01:23	CREATE VIEW EmpDeptDetailsView AS SELECT e.emp_name, d.dept_name, e.s...	0 row(s) affected
✅ 166	14:01:23	SELECT * FROM EmpDeptDetailsView LIMIT 0, 1000	5 row(s) returned
⚠ 167	14:01:42	DROP VIEW IF EXISTS ECEEmployeesView	0 row(s) affected, 1 warning(s): 1051 Unknown
✅ 168	14:01:42	CREATE VIEW ECEEmployeesView AS SELECT e.emp_id, e.emp_name, d.dept_n...	0 row(s) affected
✅ 169	14:01:42	SELECT * FROM ECEEmployeesView LIMIT 0, 1000	3 row(s) returned

5. Create a View showing department-wise employee count.

The screenshot displays the SQL Developer interface. The top pane shows the SQL script for creating a view named DeptEmpCountView. The script includes a SELECT statement that joins the employees table with the departments table to calculate the employee count per department. The bottom pane shows the results of the script execution, including a table of department names and their corresponding employee counts, and an action log with status messages.

```
2 CREATE VIEW DeptEmpCountView AS
3 SELECT d.dept_name, COUNT(e.emp_id) AS EmployeeCount
4 FROM employees e
5 JOIN departments d ON e.dept_id = d.dept_id
6 GROUP BY d.dept_id, d.dept_name;
7
8 SELECT * FROM DeptEmpCountView;
```

Result Grid

	dept_name	EmployeeCount
▶	CSE	1
	ECE	3
	EEE	1

DeptEmpCountView 56 x

Output

Action Output

#	Time	Action	Message
167	14:01:42	DROP VIEW IF EXISTS ECEEmployeesView	0 row(s) affected, 1 warning(s): 1051 Unknown table or view name
168	14:01:42	CREATE VIEW ECEEmployeesView AS SELECT e.emp_id, e.emp_name, d.dept_n...	0 row(s) affected
169	14:01:42	SELECT * FROM ECEEmployeesView LIMIT 0, 1000	3 row(s) returned
170	14:01:58	DROP VIEW IF EXISTS DeptEmpCountView	0 row(s) affected, 1 warning(s): 1051 Unknown table or view name
171	14:01:58	CREATE VIEW DeptEmpCountView AS SELECT d.dept_name, COUNT(e.emp_id) ...	0 row(s) affected
172	14:01:58	SELECT * FROM DeptEmpCountView LIMIT 0, 1000	3 row(s) returned

6. Create a View showing department-wise maximum salary.

Query 1 x

Limit to 1000 rows

```
2 CREATE VIEW DeptMaxSalaryView AS
3 SELECT d.dept_name, MAX(e.salary) AS MaxSalary
4 FROM employees e
5 JOIN departments d ON e.dept_id = d.dept_id
6 GROUP BY d.dept_id, d.dept_name;
7
8 SELECT * FROM DeptMaxSalaryView;
```

Result Grid

	dept_name	MaxSalary
▶	CSE	60000.00
	ECE	67000.00
	EEE	71000.00

DeptMaxSalaryView 57 x

Output

Action Output

#	Time	Action	Message
170	14:01:58	DROP VIEW IF EXISTS DeptEmpCountView	0 row(s) affected, 1 warning(s): 1051 Unknown
171	14:01:58	CREATE VIEW DeptEmpCountView AS SELECT d.dept_name, COUNT(e.emp_id) ...	0 row(s) affected
172	14:01:58	SELECT * FROM DeptEmpCountView LIMIT 0, 1000	3 row(s) returned
173	14:02:20	DROP VIEW IF EXISTS DeptMaxSalaryView	0 row(s) affected, 1 warning(s): 1051 Unknown
174	14:02:20	CREATE VIEW DeptMaxSalaryView AS SELECT d.dept_name, MAX(e.salary) AS ...	0 row(s) affected
175	14:02:20	SELECT * FROM DeptMaxSalaryView LIMIT 0, 1000	3 row(s) returned

7. Create a View showing departments whose average salary is greater than 50000.

Query 1

```
3 SELECT d.dept_name, AVG(e.salary) AS AverageSalary
4 FROM employees e
5 JOIN departments d ON e.dept_id = d.dept_id
6 GROUP BY d.dept_id, d.dept_name
7 HAVING AVG(e.salary) > 50000;
8
9 • SELECT * FROM DeptHighAvgSalaryView;
```

Result Grid

	dept_name	AverageSalary
▶	CSE	60000.000000
	ECE	56666.666667
	EEE	71000.000000

DeptHighAvgSalaryView58

Output

Action Output

#	Time	Action	Message
173	14:02:20	DROP VIEW IF EXISTS DeptMaxSalaryView	0 row(s) affected, 1 warning(s): 1051 Unknown table or view
174	14:02:20	CREATE VIEW DeptMaxSalaryView AS SELECT d.dept_name, MAX(e.salary) AS ...	0 row(s) affected
175	14:02:20	SELECT * FROM DeptMaxSalaryView LIMIT 0, 1000	3 row(s) returned
176	14:02:38	DROP VIEW IF EXISTS DeptHighAvgSalaryView	0 row(s) affected, 1 warning(s): 1051 Unknown table or view
177	14:02:39	CREATE VIEW DeptHighAvgSalaryView AS SELECT d.dept_name, AVG(e.salary) ...	0 row(s) affected
178	14:02:39	SELECT * FROM DeptHighAvgSalaryView LIMIT 0, 1000	3 row(s) returned

8. Create a View with employee name, salary and annual salary

Query Editor

```
1 • DROP VIEW IF EXISTS AnnualSalaryCalcView;
2 CREATE VIEW AnnualSalaryCalcView AS
3 SELECT emp_name, salary AS MonthlySalary, (salary * 12) AS AnnualSalary
4 FROM employees;
5
6 • SELECT * FROM AnnualSalaryCalcView;
```

Result Grid

	emp_name	MonthlySalary	AnnualSalary
▶	Rahul	60000.00	720000.00
	Priya	67000.00	804000.00
	Anil	55000.00	660000.00
	Sneha	71000.00	852000.00
	Kiran	48000.00	576000.00

AnnualSalaryCalcView 59

Output

Action Output

#	Time	Action	Message	
⚠	176	14:02:38	DROP VIEW IF EXISTS DeptHighAvgSalaryView	0 row(s) affected, 1 warning(s): 1051 Unknown
✓	177	14:02:39	CREATE VIEW DeptHighAvgSalaryView AS SELECT d.dept_name, AVG(e.salary) ...	0 row(s) affected
✓	178	14:02:39	SELECT * FROM DeptHighAvgSalaryView LIMIT 0, 1000	3 row(s) returned
⚠	179	14:03:00	DROP VIEW IF EXISTS AnnualSalaryCalcView	0 row(s) affected, 1 warning(s): 1051 Unknown
✓	180	14:03:00	CREATE VIEW AnnualSalaryCalcView AS SELECT emp_name, salary AS Monthly...	0 row(s) affected
✓	181	14:03:00	SELECT * FROM AnnualSalaryCalcView LIMIT 0, 1000	5 row(s) returned

9. Update an employee's salary through a simple View and verify the base table.

The screenshot shows a SQL IDE interface with a query editor, a result grid, and an action output pane.

Query Editor:

```
1  -- This updates Rahul's salary using the view from Task 1
2  UPDATE BasicEmployeeView
3  SET salary = 52000
4  WHERE emp_id = 101;
5
6  -- This checks the base table to prove the update worked
7  • SELECT * FROM employees WHERE emp_id = 101;
```

Result Grid:

	emp_id	emp_name	dept_id	salary
▶	101	Rahul	1	52000.00
*	NULL	NULL	NULL	NULL

Output:

employees 60 x

Output

Action Output

#	Time	Action	Message
✓ 178	14:02:39	SELECT * FROM DeptHighAvgSalaryView LIMIT 0, 1000	3 row(s) returned
⚠ 179	14:03:00	DROP VIEW IF EXISTS AnnualSalaryCalcView	0 row(s) affected, 1 warning(s): 1051 Unknown
✓ 180	14:03:00	CREATE VIEW AnnualSalaryCalcView AS SELECT emp_name, salary AS Monthly...	0 row(s) affected
✓ 181	14:03:00	SELECT * FROM AnnualSalaryCalcView LIMIT 0, 1000	5 row(s) returned
✓ 182	14:03:19	UPDATE BasicEmployeeView SET salary = 52000 WHERE emp_id = 101	1 row(s) affected Rows matched: 1 Changed
✓ 183	14:03:19	SELECT * FROM employees WHERE emp_id = 101 LIMIT 0, 1000	1 row(s) returned

10. Use SHOW CREATE VIEW to display the definition of a View.

The screenshot shows a database management tool interface. At the top, a toolbar contains various icons for file operations, search, and execution. Below the toolbar, a text area displays the SQL command: `1 • SHOW CREATE VIEW AnnualSalaryCalcView;`. The command is highlighted in blue. Below the text area, a "Result Grid" section shows the output of the command. The grid has four columns: "View", "Create View", "character_set_client", and "collation_connection". The first row of data shows the view name "annualsalarycalcview", its definition "CREATE ALGORITHM=UNDEFINED DEFINER='r...' utf8mb4", and the character set and collation "utf8mb4_0900_ai_ci". Below the result grid, an "Output" section is visible, showing a list of actions performed by the tool. The actions are listed in a table with columns: "#", "Time", "Action", and "Message". The actions include dropping the view, creating the view, selecting data from the view, updating data in the view, and finally showing the view definition.

View	Create View	character_set_client	collation_connection
annualsalarycalcview	CREATE ALGORITHM=UNDEFINED DEFINER='r...' utf8mb4	utf8mb4	utf8mb4_0900_ai_ci

#	Time	Action	Message
179	14:03:00	DROP VIEW IF EXISTS AnnualSalaryCalcView	0 row(s) affected, 1 warning(s): 1051 Unknown
180	14:03:00	CREATE VIEW AnnualSalaryCalcView AS SELECT emp_name, salary AS Monthly...	0 row(s) affected
181	14:03:00	SELECT * FROM AnnualSalaryCalcView LIMIT 0, 1000	5 row(s) returned
182	14:03:19	UPDATE BasicEmployeeView SET salary = 52000 WHERE emp_id = 101	1 row(s) affected Rows matched: 1 Changed
183	14:03:19	SELECT * FROM employees WHERE emp_id = 101 LIMIT 0, 1000	1 row(s) returned
184	14:03:36	SHOW CREATE VIEW AnnualSalaryCalcView	1 row(s) returned

11. List all Views in the database.

The screenshot shows a database management interface. At the top, a toolbar contains various icons for file operations, execution, and search. Below the toolbar, a SQL editor displays two lines of code: `1 SHOW FULL TABLES` and `2 WHERE Table_type = 'VIEW';`. The interface includes a 'Limit to 1000 rows' dropdown and several utility icons. Below the editor, a 'Result Grid' section shows a table with two columns: 'Tables_in_employeedb1' and 'Table_type'. The table lists several views: 'annualsalarycalcview', 'annualsalaryview', 'basicemployeeview', 'cseemployees', 'departmentsalaryview', 'deptempcountview', and 'depthighsalaryview', all of which are of type 'VIEW'. Below the result grid, an 'Output' section is visible, showing a log of database actions. The log includes the execution of the SQL commands, the creation of the 'AnnualSalaryCalcView', and the execution of the 'SHOW FULL TABLES' command, which returned 17 rows.

SQL Editor:

```
1 SHOW FULL TABLES
2 WHERE Table_type = 'VIEW';
```

Result Grid:

Tables_in_employeedb1	Table_type
annualsalarycalcview	VIEW
annualsalaryview	VIEW
basicemployeeview	VIEW
cseemployees	VIEW
departmentsalaryview	VIEW
deptempcountview	VIEW
depthighsalaryview	VIEW

Output:

#	Time	Action	Message
✓ 180	14:03:00	CREATE VIEW AnnualSalaryCalcView AS SELECT emp_name, salary AS Monthly...	0 row(s) affected
✓ 181	14:03:00	SELECT * FROM AnnualSalaryCalcView LIMIT 0, 1000	5 row(s) returned
✓ 182	14:03:19	UPDATE BasicEmployeeView SET salary = 52000 WHERE emp_id = 101	1 row(s) affected Rows matched: 1 Change
✓ 183	14:03:19	SELECT * FROM employees WHERE emp_id = 101 LIMIT 0, 1000	1 row(s) returned
✓ 184	14:03:36	SHOW CREATE VIEW AnnualSalaryCalcView	1 row(s) returned
✓ 185	14:03:54	SHOW FULL TABLES WHERE Table_type = 'VIEW'	17 row(s) returned

12. Drop a View and verify that it no longer appears in the list.

The screenshot shows the SQL Developer interface. The top toolbar includes icons for file operations, execution, and search. The SQL script editor contains the following code:

```
1 • DROP VIEW IF EXISTS BasicEmployeeView;
2   Execute the selected portion of the script or everything, if there is no selection
3   -- Run this again to prove the view is gone!
4 • SHOW FULL TABLES
5   WHERE Table_type = 'VIEW';
```

Below the script editor, the 'Result Grid' tab is active, displaying a table with two columns: 'Tables_in_employeedb1' and 'Table_type'. The table lists several views, all of which are of type 'VIEW'.

Tables_in_employeedb1	Table_type
annualsalarycalcview	VIEW
annualsalaryview	VIEW
cseemployees	VIEW
departmentsalaryview	VIEW
deptempcountview	VIEW
depthhighavgsalaryview	VIEW
deptmaxsalaryview	VIEW

The 'Output' tab is also visible, showing a list of actions and their messages. The actions include updating a row in 'BasicEmployeeView', selecting rows from 'employees', creating a view, showing full tables, dropping a view, and showing full tables again.

#	Time	Action	Message
182	14:03:19	UPDATE BasicEmployeeView SET salary = 52000 WHERE emp_id = 101	1 row(s) affected Rows matched: 1 Changed:
183	14:03:19	SELECT * FROM employees WHERE emp_id = 101 LIMIT 0, 1000	1 row(s) returned
184	14:03:36	SHOW CREATE VIEW AnnualSalaryCalcView	1 row(s) returned
185	14:03:54	SHOW FULL TABLES WHERE Table_type = 'VIEW'	17 row(s) returned
186	14:04:15	DROP VIEW IF EXISTS BasicEmployeeView	0 row(s) affected
187	14:04:15	SHOW FULL TABLES WHERE Table_type = 'VIEW'	16 row(s) returned

7. Viva / Discussion Questions

1. What is a View?

A View is a virtual table created from one or more existing tables. It does not store data itself; it stores only the SQL query.

2. Why do we use Views?

Views are used to simplify complex queries, automatically calculate columns, filter specific data, and enhance database security.

3. Does a View normally store a separate copy of the data?

Normally no; a View stores the query definition and retrieves the latest data from the underlying tables

whenever you query it.

4. What is the difference between a View and a table?

A table physically stores its own data and is created with the CREATE TABLE command. A View normally does not store data, is created with the CREATE VIEW command, and can use JOINS to combine tables.

5. Can a View be created using multiple tables?

Yes, a View can be created from one or more existing tables, often by using JOIN statements.

6. Can every View be updated?

No, it depends on the View. Some simple Views can be updated, but a View containing JOINS, GROUP BY, aggregate functions (like AVG()), DISTINCT, or other non-updatable constructs may not be directly updatable.

7. Why are Views useful for security?

They allow you to restrict access by creating Views that only show specific columns or rows instead of giving a user full access to the entire base table.

8. What is the difference between CREATE VIEW and CREATE OR REPLACE VIEW?

CREATE VIEW is used to make a brand-new View, while CREATE OR REPLACE VIEW will modify an existing View without dropping it, or create it if it does not already exist.

9. How do you display the definition of a View in MySQL?

use the command SHOW CREATE VIEW view_name;.

10. How do you remove a View?

remove a View using the command DROP VIEW IF EXISTS view_name;.

8. Common Errors and Solutions

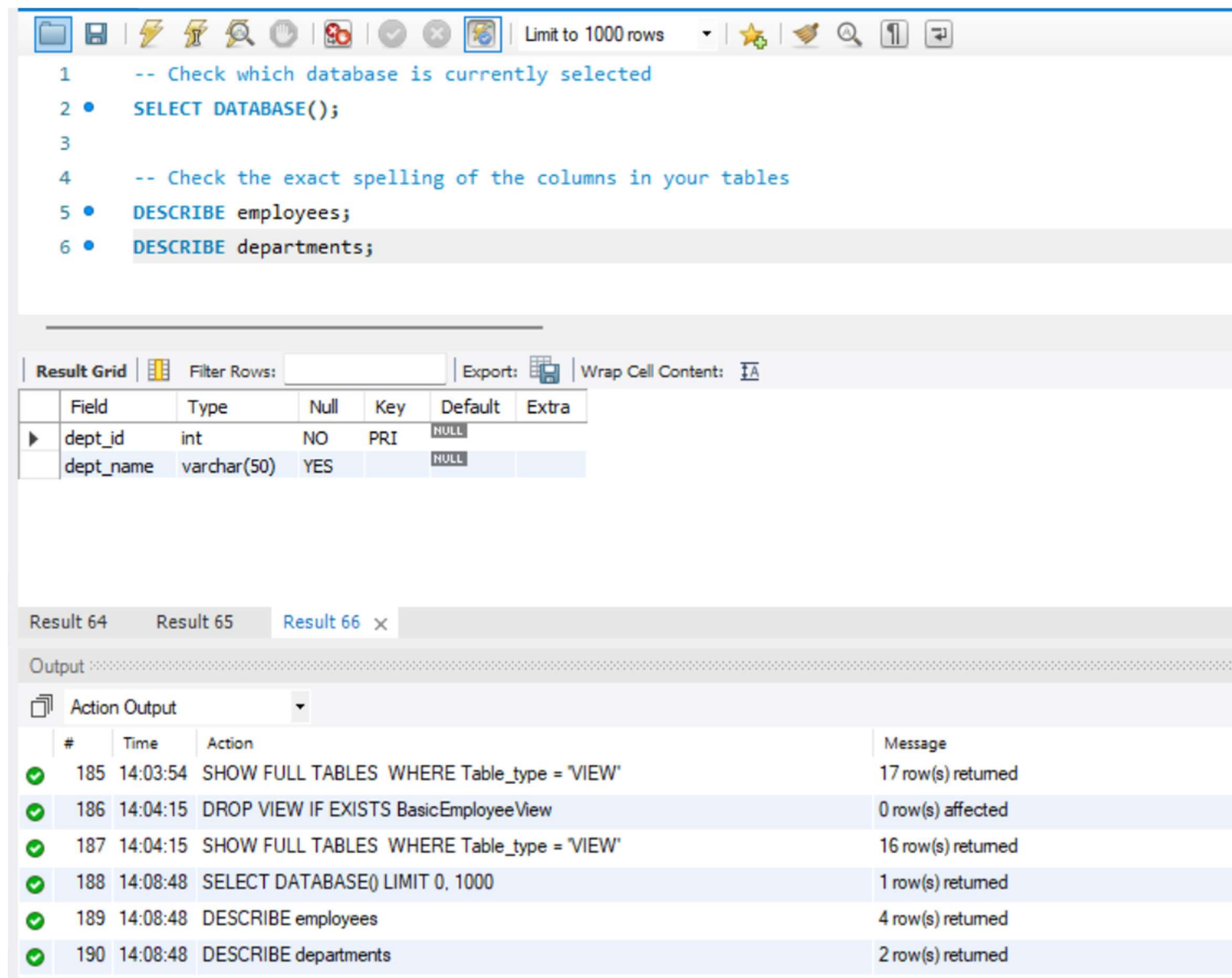
Table does not exist: Run USE employeeedb1; and check with SHOW TABLES;.

View already exists: Use DROP VIEW IF EXISTS view_name; or CREATE OR REPLACE VIEW.

Unknown column: Check the column names using DESCRIBE employees; or DESCRIBE departments;.

Cannot update View: The View may contain JOIN, GROUP BY, aggregate functions or another non-updatable construct.

Wrong database: Check the selected schema and run SELECT DATABASE();



The screenshot shows a SQL IDE interface. The script editor contains the following SQL commands:

```
1  -- Check which database is currently selected
2  •  SELECT DATABASE();
3
4  -- Check the exact spelling of the columns in your tables
5  •  DESCRIBE employees;
6  •  DESCRIBE departments;
```

Below the script editor is a results grid showing the output of the SQL commands. The grid has columns: Field, Type, Null, Key, Default, and Extra.

Field	Type	Null	Key	Default	Extra
dept_id	int	NO	PRI	NULL	
dept_name	varchar(50)	YES		NULL	

Below the results grid is an output pane showing the execution log. The log contains the following entries:

#	Time	Action	Message
✓ 185	14:03:54	SHOW FULL TABLES WHERE Table_type = 'VIEW'	17 row(s) returned
✓ 186	14:04:15	DROP VIEW IF EXISTS BasicEmployeeView	0 row(s) affected
✓ 187	14:04:15	SHOW FULL TABLES WHERE Table_type = 'VIEW'	16 row(s) returned
✓ 188	14:08:48	SELECT DATABASE() LIMIT 0, 1000	1 row(s) returned
✓ 189	14:08:48	DESCRIBE employees	4 row(s) returned
✓ 190	14:08:48	DESCRIBE departments	2 row(s) returned

9. Quick Reference

CREATE VIEW view_name AS
SELECT ...;

SELECT * FROM view_name;

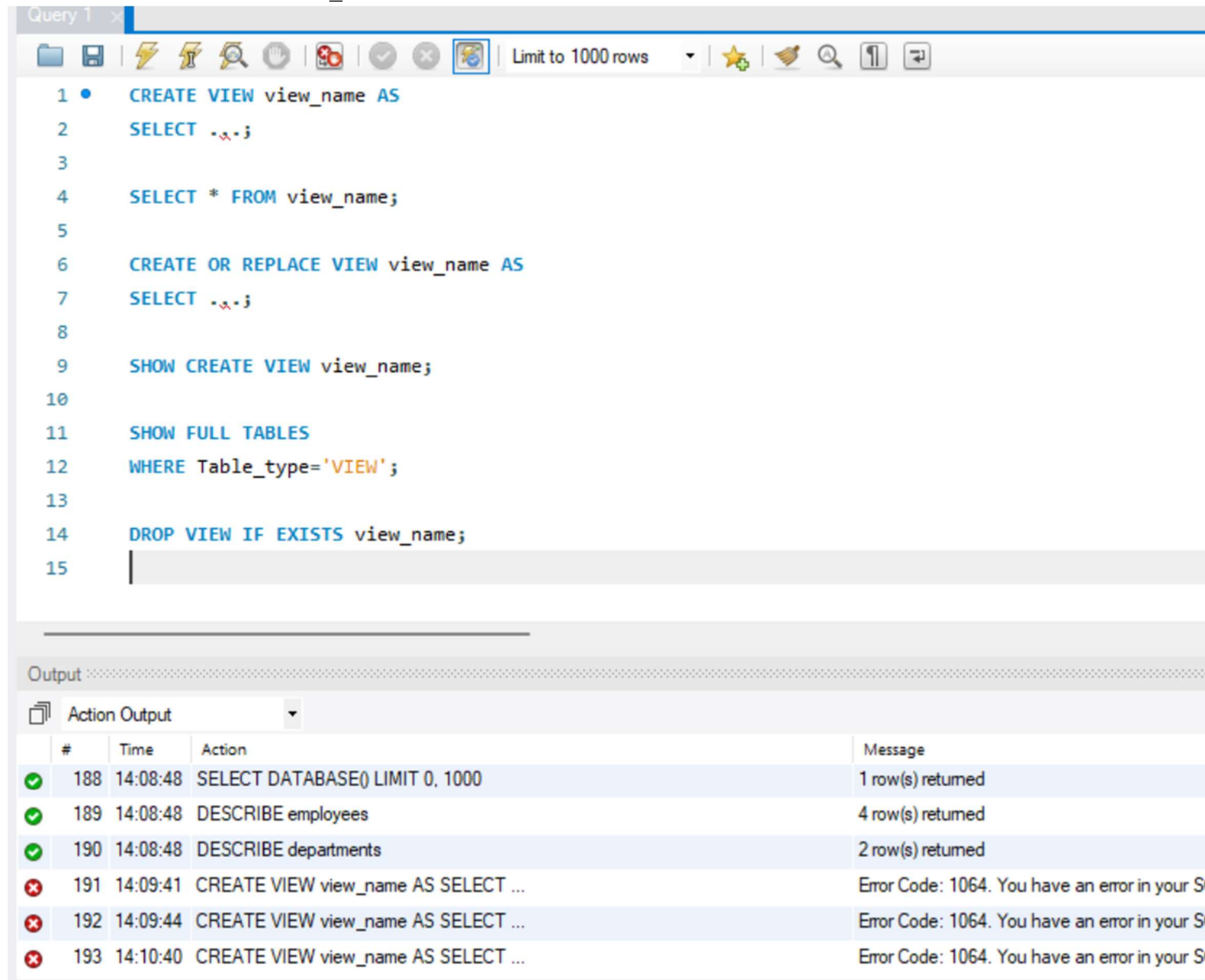
CREATE OR REPLACE VIEW view_name AS
SELECT ...;

SHOW CREATE VIEW view_name;

SHOW FULL TABLES

WHERE Table_type='VIEW';

DROP VIEW IF EXISTS view_name;



Query 1

Limit to 1000 rows

```
1 CREATE VIEW view_name AS
2 SELECT -.-;
3
4 SELECT * FROM view_name;
5
6 CREATE OR REPLACE VIEW view_name AS
7 SELECT -.-;
8
9 SHOW CREATE VIEW view_name;
10
11 SHOW FULL TABLES
12 WHERE Table_type='VIEW';
13
14 DROP VIEW IF EXISTS view_name;
15
```

Output

Action Output

#	Time	Action	Message
✓ 188	14:08:48	SELECT DATABASE() LIMIT 0, 1000	1 row(s) returned
✓ 189	14:08:48	DESCRIBE employees	4 row(s) returned
✓ 190	14:08:48	DESCRIBE departments	2 row(s) returned
✗ 191	14:09:41	CREATE VIEW view_name AS SELECT ...	Error Code: 1064. You have an error in your S
✗ 192	14:09:44	CREATE VIEW view_name AS SELECT ...	Error Code: 1064. You have an error in your S
✗ 193	14:10:40	CREATE VIEW view_name AS SELECT ...	Error Code: 1064. You have an error in your S